

[examples of usage](#)

// math

add(args: Record<string, number>) => number

multiply(args: Record<string, number>) => number

integerDivide({ arg1, arg2 }: { arg1: number, arg2: number }) => number (integer)

divide({ arg1, arg2 }: { arg1: number, arg2: number }) => number (float)

remainder({ arg1, arg2 }: { arg1: number, arg2: number }) => number

subtract({ arg1, arg2 }: { arg1: number, arg2: number }) => number

floor({ arg }: { arg: number }) => number

inc({ arg }: { arg: number }) => number

dec({ arg }: { arg: number }) => number

abs({ arg }: { arg: number }) => number

round({ arg }: { arg: number }) => number

negate({ arg }: { arg: number }) => number

// logical

logicalOR(args: Record<string, boolean>) => boolean

logicalAND(args: Record<string, boolean>) => boolean

logicalNOT({ arg }: { arg: boolean }) => boolean

// comparisons

greaterThan({ arg1, arg2 }: { arg1: number, arg2: number }) => boolean

greaterThanOrEqualTo({ arg1, arg2 }: { arg1: number, arg2: number }) => boolean

lessThan({ arg1, arg2 }: { arg1: number, arg2: number }) => boolean

lessThanOrEqualTo({ arg1, arg2 }: { arg1: number, arg2: number }) => boolean

equals({ arg1, arg2 }: { arg1: number, arg2: number } | { arg1: string, arg2: string }) => boolean

notEqual({ arg1, arg2 }: { arg1: number, arg2: number } | { arg1: string, arg2: string }) => boolean

// list

listLength({ list }: { list: array }) => number

concat(lists: Record<string, array>) => array

append({ list, element }: { list: array, element: any }) => array

intersect({ list1, list2 }: { list1: array, list2: array }) => array

listsSubtract({ list1, list2 }: { list1: array, list2: array }) => array

trueSubtract({ list1, list2 }: { list1: array, list2: array }) => array

remove exactly what is in list2 from list1, but no more.

e.g. trueSubtract(["Liberal", "Liberal", "Fascist"], ["Liberal"]) returns ["Liberal", "Fascist"].

contains({ list, element }: { list: array, element: number }) => boolean

indexOf({ list, element }: { list: array, element: number }) => number

shuffleList({ list }: { list: array }) => array

selectElement({ list, index }: { list: array, index: number }) => any (type of array element)

removeElement({ list, index }: { list: array, index: number }) => array

sublist({ list, start, end }: { list: array, start: number, end: number }) => array

returns the sublist of "list", starting at index "start" and ending at index ("end"-1) inclusive.

e.g. sublist(["Hi", "there", "it's", "me"], 1, 3) returns ["there", "it's"]

listByDictionary({ list, dict }: { list: string[], dict: Record<string, any> }) => any[]

returns a list where each string in the original list is mapped to its value in the dictionary

createList(args: Record<string, any> }) => any[]

takes any number of parameters (they must all be of the same type - string, boolean, or integer) and returns a list of those elements.

example of usage:

```
selector: 'createList',
params: [
  {
    name: 'arg1',
```

```

    type: 'preset',
    value: 'werewolf',
  },
  {
    name: 'arg2',
    type: 'preset',
    value: '2',
  },
  {
    name: 'arg3',
    type: 'preset',
    value: 'Hi!',
  },
],

```

maxIndex({ list }: { list: number[] }) => number
 return the index of the maximum element in a list of integers

minIndex({ list }: { list: number[] }) => number
 return the index of the minimum element in a list of integers

createItemList({ length, item }: { length: number, item: any }) => any[]
 creates a list of length items, where each item is the same item
 e.g. createItem(3, "hello") returns ["hello", "hello", "hello"]

unique({ list }: { list: any[] }) => any[]
 returns a list of the unique elements of the list

minValue({ list }: { list: number[] }) => number
 return the minimum value in a list of integers

maxValue({ list }: { list: number[] }) => number
 return the maximum value in a list of integers

sumAllElementsList({ list }: { list: number[] }) => number
 return sum all elements in list

reverseList({ list }: { list: number[] }) => number[]
 return reverse list

// random

randomNumber([min,] max): { min: number, max: number } => number
 Return a random integer n such that min <= n < max. min default = 0.

randomElement({ list }: { list: array }) => any (type of array element)

randomElementsList({ list, length }: { list: array, length: number }) => array

// string

formatString({ format, args }: { format: string; args: string[] }) => string

listToString({ list, [readable], [delimiter] }: { list: array, readable: boolean; delimiter: string })
=> string

if readable 'false' (default) returns the array elements are separated with the delimiter (comma (",") by default).

if readable 'true': returns the following:

- If list has 1 element [el1], return "el1"
- If list has 2 elements [el1, el2], return "el1 and el2"
- If list has 3+ elements [el1, el2, el3, ...elN], return "el1, el2, el3, ... and elN" (i.e. the last word gets an "and" before it)

toLowerCase({ text: string }) => string

stringLength({ text: string }) => number

toUpperCase({ text: string }) => string

substring({text, start, end}) => string

stringToList({text}) => string[]

stringToInt({ text: string }) => number
parses text into an integer (throws if not a valid integer)

isRealEnglishWord({text}) => boolean

// conditional

ifElse({ condition, thenValue, elseValue }: { condition: boolean; thenValue: string | number; elseValue: string | number }) => string | number

// object

getObjectField({ obj, field, defaultValue}: { obj: Record<string, any>; field: string | number, defaultValue?: any }) => any (type of object field value or defaultValue if the field isn't in the object)

getObjectFieldList({ obj, field, defaultValue}: { obj: Record<string, any>; field: string | number, defaultValue?: any }) => any[] (type of object field value or defaultValue if the field isn't in the object)

getObjectKeys({ obj }: { obj: Record<string, any> }) => string[]

getObjectValues({ obj }: { obj: Record<string, any> }) => any[]

createDict({ keys, values }: { keys: string[], values: any[] }) => Record<string, any>

// others

polynomial({ value, coefficients }: { value: number, coefficients: number[] }) => number

a selector for evaluating a polynomial with coefficients and input value

- a. Input value is a number x
- b. Coefficients is a list $[c_0, c_1, c_2, \dots, c_{(N-1)}]$
- c. Returns $c_0 + c_1 * x + \dots + c_{(N-1)} * x^{(N-1)}$

// common state selectors

allPlayers() => string[] (returns ids of users who isPlayer)

allPlayersByOrder() => string[] (returns ids of users who isPlayer in the order they currently appear on the screen)

allConnectedUsers() => string[] (returns ids of all connected users)

getPlayerNameById({ id }: { id: string }) => string (returns displayName by userId)

getPlayerNamesByIds({ ids }: { ids: string[] }) => string[] (returns displayNames by userIds)

getRemainingTimer({ timerId }: { timerId: string }) => number

getHostPlayerId() => string | undefined

nextPlayer({ playersList, playerId}: { playersList: string[]; playerId: string }) => string;
returns the next player id from the players list after given playerId. Returns the first element for the last one.

prevPlayer({ playersList, playerId}: { playersList: string[]; playerId: string }) => string;
returns the prev player id from the players list before given playerId. Returns the first element for the last one.

allSpectators() => string[] (returns ids of spectators)

allUsers() => string[] (returns ids of all users: players and spectators)

allHumanPlayers() => string[] (returns ids of all human players)

allRobotPlayers() => string[] (returns ids of all bot players)

//cache selectors

incObjectFieldValue({ ids, objectName }: { ids: string[]; objectName: string }) => any

decObjectFieldValue({ ids, objectName }: { ids: string[]; objectName: string }) => any

getCachedValue({ name }: { name: string }) => any

getCachedObjectValue({ objectName, value, defaultValue }: { objectName: string; value: string, defaultValue?: any }) => any (type of object field value or defaultValue if the field isn't in the object)

setCachedObjectFieldValue(

{
 objectName,
 fieldName,
 value,
}: { objectName: string; fieldName: string; value: unknown }) => object

sets the value of the field in the object, and returns the object.

if there is no such object in the cache, it is created with this field

setCachedObjectFieldsValues(

{
 objectName,
 fieldsNames,
 values,
}: { objectName: string; fieldsNames: string[]; values: unknown[] }) => object

takes a list of keys and a list of values, and updates each key-value pair in the object.

If there is no such object in the cache, it is created with these fields

isCachedObjectContainsField(

{ objectName, fieldName }: { objectName: string; fieldName: string }) => boolean

return true or false depending on whether the field in the object

// other

selectRandomPuzzgridId() => string;

getObjectKeysWithPositiveValue({ data }: { data: Record<string, number> }) => string[]